04 从业务术语到机器可理解的世界模型：类、实体、属性、关系、规则与动作

陈乔 通往AI之桥 2026年10月25日 19:48

通往AI之桥 | 本体论在 AI 工程中的实践系列 · 第 04 篇：把“大家都懂”的业务语言，变成系统和 Agent 都能使用的语义契约

> 一套本体真正开始发挥作用，不是因为我们画出了一张漂亮的关系图，而是因为不同的人、系统和 Agent 对关键业务对象有了共同、明确、可验证的理解。

一、为什么 AI 工程需要一份“世界模型”？

在企业软件里，有些词看起来人人都懂：需求、产品、组件、版本、测试、发布、部署。

但当我们把它们交给多个 AI Agent 协作时，问题就出现了。

开发 Agent 说“版本已生成”，可能指代码已经提交；测试 Agent 说“版本通过”，可能指测试环境里某个构建通过了回归测试；发布 Agent 说“版本已发布”，可能指制品已进入仓库；客户成功团队说“已交付”，却可能意味着软件已经部署到客户环境，并完成验收。

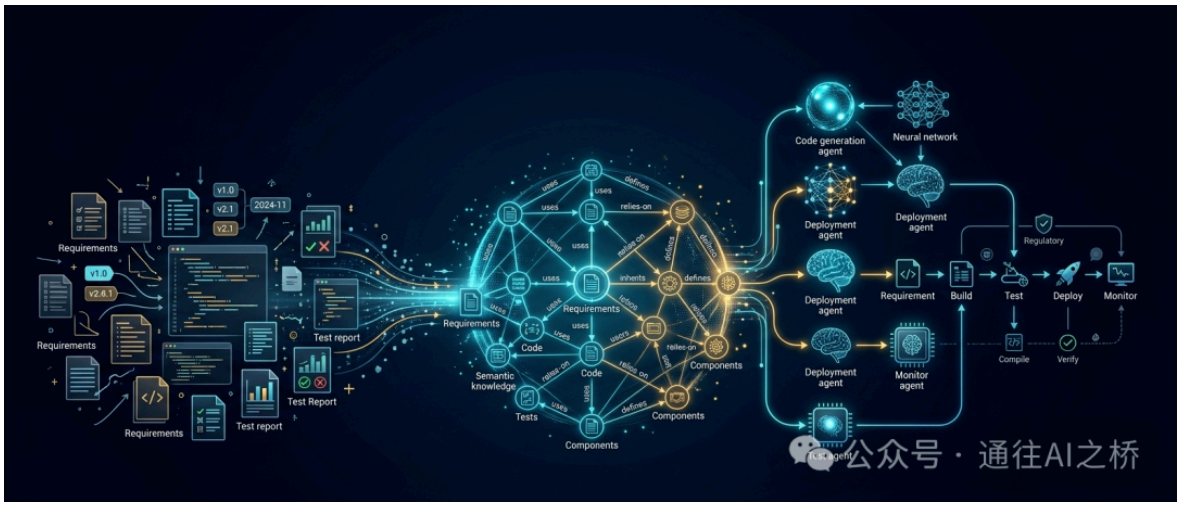
这些表达都很自然，但它们不一定指向同一个事实。

如果各个 Agent 只通过自然语言交流，系统就可能在“看起来理解了”的情况下，把提交当成构建、把构建当成发布、把发布当成客户交付。

这不是模型是否足够聪明的问题，而是业务语义没有被明确表达的问题。

> **本体工程要做的事情之一，就是把隐含在日常语言、文档和人员经验里的业务含义，转成可共享、可复用、可检查的模型。**

本文以 AI 软件工厂为例，讨论类、实例、属性、关系、规则、事件和动作这些基本构件。它们并非只用于画本体图，而是可以逐步形成需求管理、代码生成、测试、发布和运维 Agent 共同遵循的语义契约。



【图 1：从模糊业务语言到机器可理解的世界模型】

二、先区分七个容易混淆的概念

本体建模常见的第一个误区，是直接开始画关系图，却没有先说明每个词究竟代表什么。下面这些概念看起来基础，却决定了模型能否被正确使用。

1. 概念：我们在业务上“谈论的是什么”

“软件组件”“产品版本”“测试结果”都是业务概念。概念可以先以自然语言、术语表和示例表达，它不一定一开始就被编码成正式的本体类。

概念定义至少应回答：它是什么意思、不包括什么、与相近概念有什么区别，以及由谁负责维护。

例如，“发布（Release）”不能只写成“软件发布”。它究竟指一组准备交付的制品，还是把制品部署到生产环境的动作？这两个含义应当分开。

2. 类 / 类型（Class / Type）：同一类事物的共同定义

当我们决定把某类对象用统一结构来描述时，就形成了类或类型。

例如：Product（产品）、Component（组件）、Build（构建）、TestRun（测试运行）、Release（发布版本）、Deployment（部署记录）。

一个类通常会定义对象的身份方式、属性、可关联的关系，以及适用的约束。不同的建模技术对“类”的形式化能力并不完全相同；在 OWL 中，类有严格的形式语义；在业务平台中，Object Type 或 Entity Type 也可能承担类似角色，但未必等同于 OWL 类。

3. 实体 / 实例（Entity / Individual）：现实中具体的那一个

Product 是一种类型；“AI 软件工厂平台”可以是一个产品实例。ComponentVersion 是一种类型；“Kafka 3.7.2 的某个发行构建”则是一个具体实例。

这里需要留意术语差异：在数据建模和业务分析中，“实体”有时指实体类型，有时指具体对象。为了避免歧义，本文尽量使用“类 / 类型”指一类事物，使用“实例 / 个体”指某一个具体对象。

实例必须有可识别的身份。不能仅凭两个对象的显示名称相同，就断定它们是同一个对象。比如，一个名为 3.7.2 的字符串可能出现在多个组件上；只有结合组件名、发行来源、制品摘要等信息，才能确定具体版本对象。

4. 属性（Property）：对象自身有什么特征

属性描述对象的特征，例如：

- 组件的名称、语言、许可证类型；
- 版本的版本号、制品摘要、发布时间；
- 测试运行的开始时间、结束时间和执行状态；
- 部署记录的目标环境、执行人和结果。

工程上常见的属性可分为两类：

- **数据属性：** 把对象连接到一个值，例如版本号是 `3.7.2`，测试状态是 `PASSED`。
- **对象关系：** 把一个对象连接到另一个对象，例如构建使用某个组件版本，部署属于某个客户环境。

在 RDF/OWL 语境中，数据属性和对象属性有明确的建模区别；在其他平台中，具体术语可能不同。[1][2]

5. 关系（Relationship）：对象之间有什么联系

关系往往是本体最有价值的部分，因为很多业务问题并不只依赖某个对象的属性，而依赖对象之间的连接。

例如：

- 一个需求由一个或多个变更实现；
- 一个构建由某次代码提交产生；
- 一次测试验证某个构建；
- 一个发布包包含若干制品；
- 一次部署把某个发布包部署到某个客户环境；
- 一个安全发现影响特定组件版本。

关系最好具有明确的方向、含义和基数预期。例如，“部署使用发布包”与“发布包被部署到环境”表达的事实相关，但在建模时应决定是只存一条关系、反向查询，还是显式维护两条关系，避免数据不一致。

6. 规则 / 约束（Rule / Constraint）：什么条件必须成立

规则让本体不止描述“世界里有什么”，还可以表达对对象、关系或状态的要求。

例如：

- 一个生产发布必须关联可追踪的构建；
- 强制回归测试未通过的构建不能进入发布审批；
- 生产部署必须关联已批准的发布包；
- 高危漏洞未修复或未经过授权豁免时，不得发布。

这里有一个重要边界：不是所有规则都适合只用 OWL 表达。OWL 适合表达具有形式语义的类别与关系知识，并支持特定类型的逻辑推理；SHACL 等约束语言则可以用于检查 RDF 图是否满足指定的数据形状和约束；复杂的时序、审批、权限或运行时策略，通常还要交给策略引擎和工作流系统执行。[2][3]

因此，建模时最好标明规则类型：它是分类与推理规则、数据质量约束、业务不变量，还是动作执行前的策略条件。

7. 事件与动作：发生了什么，和“要做什么”不是一回事

这两个概念在多 Agent 系统里尤其重要。

事件（Event）描述已经发生或被系统观察到的事实，例如：

- `BuildCompleted`：构建已完成；
- `TestFailed`：测试失败；
- `CriticalFindingDetected`：发现高危安全问题；
- `ReleaseApproved`：发布已获批准；
- `DeploymentFailed`：部署失败。

动作（Action）描述某个主体可以请求或执行的操作，例如：

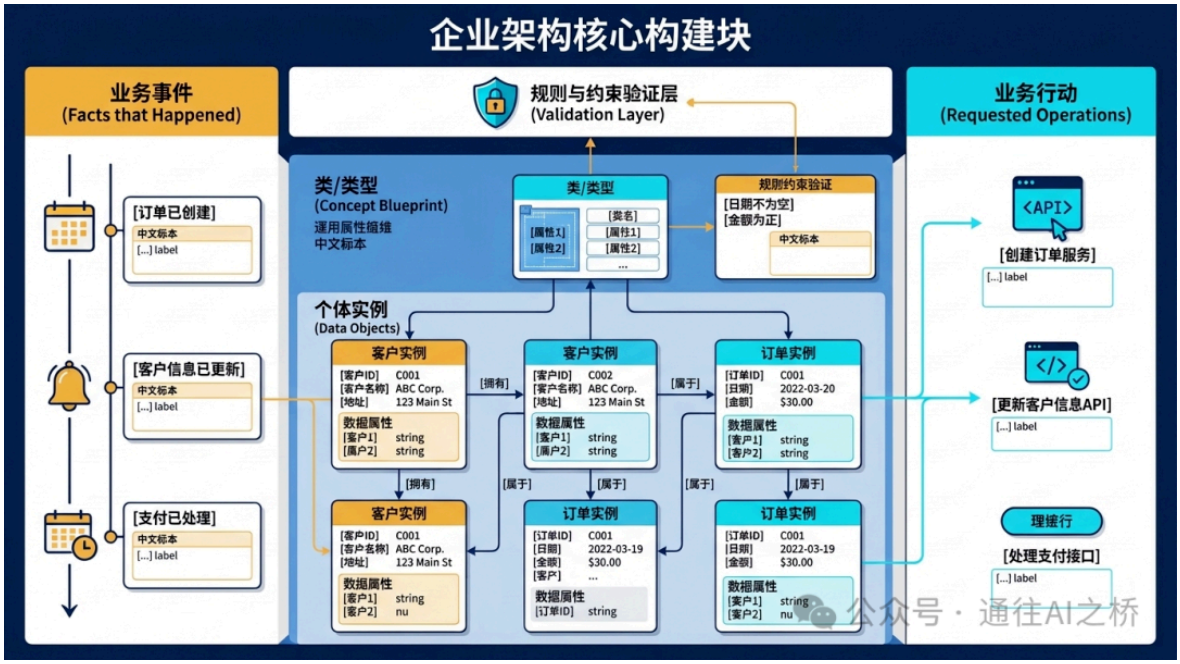
- `RunTests`

：运行测试；

- `CreateRelease`
  ：创建发布包；
- `ApproveRelease`
  ：批准发布；
- `DeployRelease`
  ：部署到目标环境；
- `RollbackDeployment`
  ：回滚部署。

事件记录事实，动作试图改变状态。两者不能混为一谈：Agent 说“我准备部署”并不意味着部署已经发生；工具返回“部署成功”后，系统还应产生有证据支持的部署结果事件。

同样，本体可以描述动作的含义、输入、输出、前置条件与相关对象，但并不意味着本体语言本身就会执行这个动作。真正执行操作的仍是 API、工具、工作流引擎或运行时系统；权限、审批、幂等、回滚和审计也需要独立的工程机制。



【图 2：类、实例、属性、关系、规则、事件与动作的概念关系图】

三、从术语表到语义契约：本体建模的四个层次

在实践中，不必一上来就设计一个庞大、形式化的企业本体。更实际的做法，是把建模分成四个逐步加深的层次。

第一层：术语表——先统一“词是什么意思”

例如，明确“发布包”是可交付的软件制品集合，“部署”是把指定制品安装或启用到某个目标环境的操作，“部署记录”则是描述某次部署的对象。

术语表适合初期统一语言，但通常无法充分表达复杂关系和机器可验证规则。

第二层：领域模型——明确对象和关系

在术语表基础上，定义类、关键属性、关系及核心标识。例如：

Requirement → Change → Build → TestRun → Release → Deployment

这条链把需求、代码变更、构建、测试、发布与部署联系起来，便于进行追踪和影响分析。

第三层：语义契约——明确数据含义和边界

为属性、关系和状态增加数据类型、枚举、必填要求、基数、来源系统、时间含义和业务约束。例如，“测试通过”必须能追溯到测试套件版本、被测构建、执行环境及报告。

这时，本体不再只是一张图，而逐渐成为系统集成、数据校验和 Agent 协作可以共同使用的契约。

第四层：可执行工作流——让模型连接工具与状态变化

最后，明确哪些动作可用、需要什么输入、前置条件是什么、成功或失败如何表示、由谁授权，以及事件如何回写到系统。工作流引擎和工具系统根据这些定义进行执行。

需要强调的是，这四层不必由一个技术产品全部实现。术语表可能在目录或元数据平台里维护，领域模型使用 RDF/OWL 或属性图表达，约束由 SHACL 或业务规则引擎验证，动作由 API 和工作流执行。关键在于语义要一致、版本要可追踪、责任要明确。

四、一个可评审的本体草图：AI 软件工厂如何定义“发布”

下面做一个小型示例。它不是完整的 OWL 本体，而是一份便于产品经理、架构师、开发、测试和安全人员共同评审的领域草图。

1. 核心类 / 类型

类 / 类型	含义	关键属性示例
`Requirement`	需要实现或解决的需求	ID、标题、优先级、验收标准
`Component`	可独立识别和复用的软件组件	名称、类型、维护团队、许可证
`ComponentVersion`	某组件的一个具体版本或制品版本	版本号、制品摘要、来源、创建时间

类 / 类型	含义	关键属性示例
Build	由特定源代码和构建配置产生的一次构建	提交 ID、构建环境、状态、时间
TestRun	针对指定对象执行的一次测试运行	测试套件、结果、报告地址、时间
SecurityFinding	一条安全或合规发现	严重级别、状态、证据、处理期限
Release	准备交付的一组经过标识的软件制品	发布版本、说明、批准状态
Deployment	将发布内容部署到指定环境的一次操作记录	环境、执行时间、结果、操作者
Environment	运行软件的目标环境	环境类型、区域、配置标识
Customer	订购或使用软件产品的客户主体	客户 ID、组织名、合同标识

2. 核心关系

- `Requirement implementedBy Change`
  ：需求由哪些代码或配置变更实现。
- `Build builtFrom Commit`
  ：构建由哪个提交产生。
- `Build uses ComponentVersion`
  ：构建使用了哪些组件版本。
- `TestRun validates Build`
  ：测试运行验证哪个构建。
- `SecurityFinding affects ComponentVersion`
  ：安全发现影响哪个组件版本。
- `Release includes Build`
  ：发布包含哪些构建或制品。
- `Deployment deploys Release`
  ：部署操作部署了哪个发布包。
- `Deployment targets Environment`
  ：部署的目标环境是什么。
- `Environment belongsTo Customer`
  ：运行环境属于哪个客户。

3. 核心约束与规则

- 每个发布包必须具有唯一、可追踪的发布标识。
- 每个生产部署必须引用一个明确的发布包和目标环境。
- 每个发布包都应能追溯到对应的构建制品、源代码提交和测试结果。

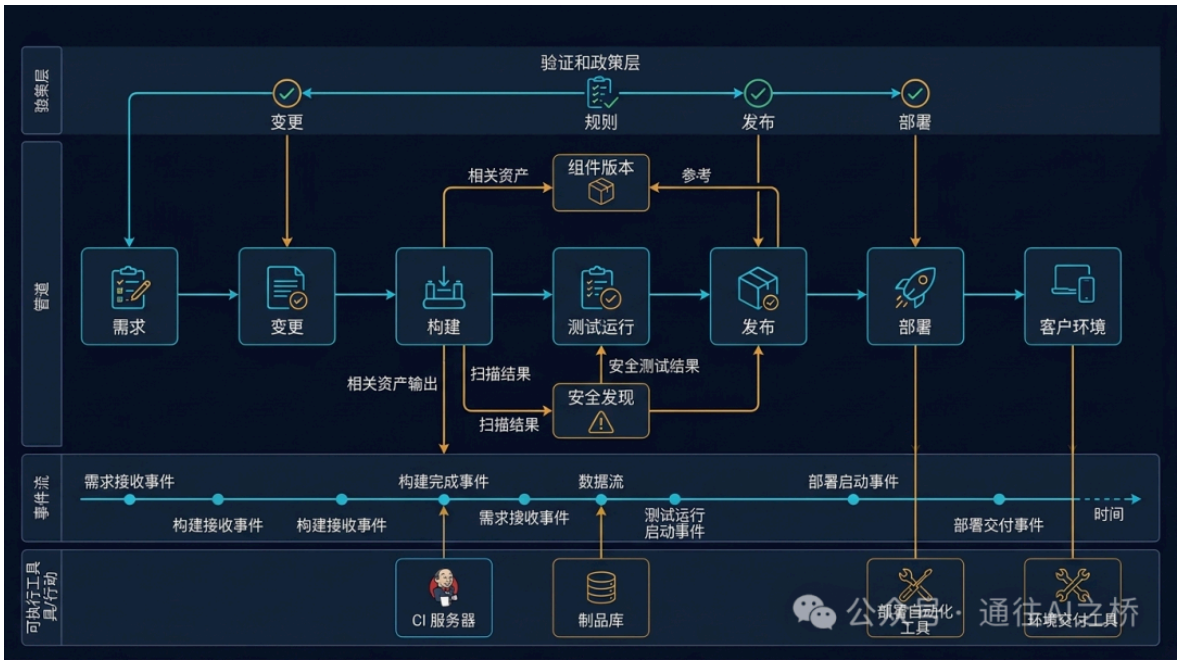
- 必需的测试未通过时，发布不能进入批准状态；若确实需要豁免，必须记录授权人、理由、范围和期限。
- 存在未处理的高危安全问题时，系统应按安全政策阻断发布或进入明确的例外审批流程。

注意，最后两条属于需要工作流或策略引擎执行的业务控制规则，并非仅把一句话写进本体就能保证它们被遵守。

4. 事件与动作

类型	示例	含义
事件	`BuildCompleted`	某次构建已完成，带有构建 ID 和结果
事件	`TestFailed`	某个构建未通过指定测试
事件	`CriticalFindingDetected`	检测到高危问题，并关联受影响版本
事件	`ReleaseApproved`	某发布包已在授权流程中被批准
动作	`RunTests`	对指定构建执行测试
动作	`CreateRelease`	根据符合条件的制品创建发布包
动作	`ApproveRelease`	在权限和条件满足时审批发布
动作	`DeployRelease`	将批准的发布包部署到目标环境
动作	`RollbackDeployment`	将环境回退到已知的目标版本

这份草图的价值，不在于类的数量，而在于它可以被不同角色拿来讨论：测试负责人能确认“测试通过”应具备哪些证据，安全负责人能定义漏洞豁免规则，运维负责人能明确部署和回滚状态，Agent 工程师则能把这些定义映射到工具输入输出和工作流状态。



【图 3：AI 软件工厂本体草图】

五、怎样避免把本体做成“漂亮但无用的图”？

本体模型常见的失败，不是工具不够强，而是没有将模型绑定到实际工程问题。下面几种反模式尤其值得注意。

反模式一：把数据库表直接改名成类

数据库表反映某个系统当前的存储设计，不一定等于稳定的业务概念。不同系统可能用不同表表达同一概念；同一张表也可能混合多个概念。应先理解业务定义，再建立到物理字段的映射。

反模式二：只画概念，不写边界

如果只定义“Release 是发布”，却不说它与 Build、Deployment 有什么区别，多个 Agent 还是会各自解释。关键概念需要定义边界、身份、关系、状态及反例。

反模式三：把所有逻辑都放进本体

本体适合表达稳定的概念和关系，以及适合形式化描述的约束；高频业务流程、长事务、权限审批、重试和回滚通常还需要工作流引擎、策略引擎和普通应用代码。不要为了“本体驱动”而把整个系统强行写成逻辑公理。

反模式四：把事件和动作混为一谈

“发布成功”是事实事件；“执行发布”是动作请求。二者在数据结构、权限、审计和重试语义上不同。如果 Agent 只生成了动作请求，系统不能提前把状态标记为成功。

反模式五：模型没有负责人和版本管理

本体会变化。产品版本命名、部署状态和安全政策可能随着业务演进而调整。应明确领域负责人、变更审批、兼容策略、迁移规则和回归测试，并将本体版本与数据映射、Agent、工具和工作流版本关联起来。

六、从哪里开始？一份轻量的实践清单

如果你准备在某个 AI 项目中引入本体，可以从一个两周左右的概念验证开始，而不是先启动覆盖全企业的大工程。

第一步：选一个明确的问题。例如“哪些客户部署受到某个漏洞影响”，或者“为什么某次发布没有通过验收”。避免一开始就提出“建立企业统一世界模型”这样无法验收的目标。

第二步：列出核心概念。控制范围，先识别 8—15 个关键类 / 类型，定义每个概念的含义、身份和边界。

第三步：画出必要关系。只保留回答目标问题所需要的关系，并标明关系的方向、来源、时间含义和可信度。

第四步：写出规则和反例。除了正常路径，还要列出缺字段、状态冲突、版本不明、测试失败、权限不足等异常情形。

第五步：映射真实数据。将业务模型连接到 API、数据库字段、代码仓库、制品库或日志系统。记录来源和更新机制，不要让人工维护的图长期偏离真实系统。

第六步：定义验证问题。用一组实际问题检验模型是否有用，并记录查询正确率、关系完整率、人工修正次数、Agent 执行失败率和处理时间等指标。

第七步：再决定技术实现。小范围模型可以从术语表和结构化 JSON 开始；需要 RDF 互操作或形式语义时可评估 RDF/OWL；需要检查图数据约束时可评估 SHACL；需要路径查询时再选择 SPARQL 或属性图查询。技术选择应由问题驱动。



【图 4：从一个业务问题启动本体建模的七步路线图】

七、结语：让业务语言成为工程系统可以共同遵守的契约

从业务术语到机器可理解的世界模型，并不是一步完成的。它可以从一个词汇表开始，逐渐发展为领域模型、语义契约，最终与数据映射、规则校验、工具调用和工作流连接起来。

类帮助我们说明“一类事物是什么”；实例指向“具体的哪一个对象”；属性描述对象自身的特征；关系说明对象如何连接；规则定义哪些条件必须满足；事件记录发生过什么；动作则描述系统可以请求或执行什么操作。

当这些定义足够清楚、经过业务专家审阅，并与真实数据和工程工具建立联系时，本体才真正从概念图变成可使用的工程资产。

对 AI 工程来说，这一点尤其重要。我们希望 Agent 能理解业务、调用工具并推进工作，但不能只依赖它临场猜测业务含义。越是跨系统、跨团队、跨 Agent 的复杂流程，越需要可追踪的对象、明确的关系、可验证的约束和清晰的动作边界。

> **本体不是让机器“看起来懂业务”，而是把业务中最重要的含义、关系和规则，变成机器可以共同使用、检查和持续维护的契约。**

系列预告

下一篇将进入本体工程方法论：**《本体的“星际导航”：如何从混沌业务中识别和定义领域本体？》**我们将从业务文档、SQL、API、事件日志和专家访谈出发，讨论怎样发现核心概念、处理同名异义与异名同义，并把 AI 作为建模助手，而不是让模型无监督地决定业务定义。

参考资料

1. W3C. OWL 2 Web Ontology Language Primer (Second Edition)：介绍 OWL 中的类、个体、属性和形式语义。
2. W3C. RDF 1.1 Concepts and Abstract Syntax：介绍 RDF 图、资源与三元组表示。
3. W3C. Shapes Constraint Language (SHACL)：用于检查 RDF 图是否满足指定形状和约束。
4. W3C. SPARQL 1.1 Query Language：RDF 图的查询语言。
5. Palantir. Ontology Overview：以对象、属性、链接及动作/函数组织企业语义与操作能力的产品路线示例。
6. Gruber, T. R. A Translation Approach to Portable Ontology Specifications：早期知识工程中关于本体规范的经典论文。

收录于 本体论在AI工程中的实践
作者提示: 内容由AI生成